

AerolineaAPI

CRUD completo · Relaciones · DTOs · ResponseEntity · Validaciones · Swagger

En la sesión anterior construiste la base de AerolineaAPI: las entidades **Vuelo** y **Pasajero**, sus repositorios, servicios con **findAll()** y **save()**, y un endpoint GET básico para cada una. Hoy completas el sistema. Al terminar tendrás una API con CRUD completo, relaciones entre entidades, serialización correcta, validaciones activas y documentación Swagger.

Lo que ya tienes funcionando

```
model/
  EstadoVuelo.java      ← enum: PROGRAMADO · EN_VUELO · ATERRIZADO · CANCELADO
  Vuelo.java            ← @Entity con getters y setters
  Pasajero.java         ← @Entity con getters y setters
repository/
  VueloRepository.java  ← extends JpaRepository – cuerpo vacío
  PasajeroRepository.java ← extends JpaRepository – cuerpo vacío
service/
  VueloService.java     ← findAll() y save()
  PasajeroService.java  ← findAll() y save()
controller/
  VueloController.java  ← @GetMapping básico
  PasajeroController.java ← @GetMapping básico
```

Lo que construyes hoy

#	Qué construyes	Qué queda funcionando
1	CRUD completo de Vuelo y Pasajero	Crear, leer, actualizar y eliminar vuelos y pasajeros desde Postman
2	Entidad Reserva + enum ClaseAsiento	La entidad que vincula un pasajero con un vuelo
3	DTOs para Reserva	Serialización sin ciclos, entrada y salida limpias
4	ResponseEntity en los cuatro controllers	Códigos HTTP correctos: 200, 201, 204, 404
5	Validaciones con Bean Validation	Datos inválidos rechazados con 400 antes de tocar la BD
6	Swagger / OpenAPI	Documentación visual interactiva en /swagger-ui.html

01 CRUD completo — Vuelo y Pasajero

Los servicios actuales solo tienen **findAll()** y **save()**. Para tener CRUD completo necesitas agregar **findById()**, **update()** y **delete()**. El controller necesita los cinco endpoints HTTP correspondientes. Sigue el mismo patrón para ambas entidades.

VueloService — métodos a agregar

Método	Qué recibe	Qué hace	Qué retorna
findById(Long id)	id del vuelo	Busca en el repositorio. Si no existe retorna null	Vuelo
update(Long id, Vuelo datos)	id + objeto con nuevos valores	Busca el vuelo existente, actualiza sus campos, guarda	Vuelo
delete(Long id)	id del vuelo	Elimina por ID. Sin retorno	void

■ En update(), los campos a actualizar son: origen, destino, fechaHora y estado. Si findById() retorna null, update() también retorna null — el controller se encarga de responder 404.

PasajeroService — métodos a agregar

Misma estructura que VueloService. Los campos a actualizar en **update()** son: nombre, apellido, documento y email.

VueloController — endpoints a agregar

Anotación	Ruta	Qué hace
@GetMapping("/{id}")	/vuelos/{id}	Retorna el vuelo con ese ID
@PostMapping	/vuelos	Crea un vuelo nuevo
@PutMapping("/{id}")	/vuelos/{id}	Actualiza un vuelo existente
@DeleteMapping("/{id}")	/vuelos/{id}	Elimina un vuelo por ID

PasajeroController sigue el mismo patrón con la ruta **/pasajeros**.

Verificar antes de continuar

Correr la app. Desde Postman ejecutar este flujo para cada entidad:

Operación	Endpoint de ejemplo	Respuesta esperada
Crear un vuelo	POST /vuelos	El objeto creado con id asignado
Leer todos	GET /vuelos	Array con el vuelo creado
Leer uno	GET /vuelos/1	Solo ese vuelo

Actualizar	PUT /vuelos/1	El vuelo con los nuevos valores
Eliminar	DELETE /vuelos/1	Sin cuerpo
Verificar eliminación	GET /vuelos	Array vacío []

■ Body de ejemplo para POST /vuelos: { "origen": "BOG", "destino": "MDE", "fechaHora": "2026-06-01T08:00:00", "estado": "PROGRAMADO" }

■ Body de ejemplo para POST /pasajeros: { "nombre": "Laura", "apellido": "Martínez", "documento": "10203040", "email": "laura@aerolinea.com" }

✓ CRUD de Vuelo y Pasajero funcionando en Postman — continuar con Tarea 2

02 Entidad Reserva y enum ClaseAsiento

Una reserva representa el acto de un pasajero de tomar asiento en un vuelo. Tiene fecha de reserva, clase de asiento, y dos relaciones: pertenece a un **Pasajero** y a un **Vuelo**. Esas relaciones son **@ManyToOne** — muchas reservas pueden pertenecer al mismo pasajero, y muchas reservas pueden estar en el mismo vuelo.

ClaseAsiento.java — crear en el paquete model

Enum con tres valores: **ECONOMICA**, **EJECUTIVA**, **PRIMERA_CLASE**. Misma estructura que EstadoVuelo.

Reserva.java — crear en el paquete model

Campo	Tipo	Anotaciones JPA
id	Long	@Id · @GeneratedValue(IDENTITY)
fechaReserva	LocalDateTime	@Column(nullable = false)
clase	ClaseAsiento	@Enumerated(EnumType.STRING)
pasajero	Pasajero	@ManyToOne · @JoinColumn(name = "pasajero_id", nullable = false)
vuelo	Vuelo	@ManyToOne · @JoinColumn(name = "vuelo_id", nullable = false)

■ Reserva necesita: constructor vacío (obligatorio para JPA), constructor con todos los campos excepto id, y getters y setters para todos los campos.

■ Vuelo y Pasajero NO necesitan lista de reservas (@OneToMany). La relación solo vive en Reserva. Esto evita ciclos de serialización sin necesidad de anotaciones adicionales en los modelos existentes.

ReservaRepository — crear en el paquete repository

Interface que extiende **JpaRepository<Reserva, Long>**. Anotada con **@Repository**. Cuerpo vacío por ahora.

✓ Reserva creada — Hibernate debe crear la tabla reserva al reiniciar la app

Verificar en pgAdmin que la tabla **reserva** aparece con las columnas **id**, **fecha_reserva**, **clase**, **pasajero_id** y **vuelo_id**.

03 DTOs para Reserva

Reserva tiene relaciones **@ManyToOne** con Pasajero y Vuelo. Si expones la entidad directamente, Jackson intentará serializar el Pasajero completo, luego el Vuelo completo — y si esos tuvieran listas de Reserva, entraría en un ciclo infinito. La solución es usar DTOs: un objeto de entrada que recibe solo IDs, y un objeto de salida con los datos que el cliente necesita ver.

Crear el paquete dto si no existe

Clic derecho sobre el paquete raíz → New → Package → **dto**

ReservaRequestDTO — crear en el paquete dto

Lo que el cliente envía al crear una reserva. En lugar de enviar objetos completos, envía solo los IDs de referencia.

Campo	Tipo	Descripción
fechaReserva	LocalDateTime	Fecha y hora de la reserva
clase	ClaseAsiento	Clase del asiento
pasajeroId	Long	ID del pasajero que reserva
vueloId	Long	ID del vuelo reservado

■ ReservaRequestDTO no tiene @Entity, @Id ni ninguna anotación JPA. Es una clase Java simple con constructor vacío y getters y setters.

ReservaResponseDTO — crear en el paquete dto

Lo que el cliente recibe al consultar una reserva. Incluye datos del pasajero y del vuelo para que el frontend no necesite hacer peticiones adicionales.

Campo	Tipo	De dónde viene
id	Long	reserva.getId()
fechaReserva	LocalDateTime	reserva.getFechaReserva()
clase	ClaseAsiento	reserva.getClase()
pasajeroId	Long	reserva.getPasajero().getId()
pasajeroNombre	String	reserva.getPasajero().getNombre() + " " + getApellido()

vueloId	Long	reserva.getVuelo().getId()
vueloOrigen	String	reserva.getVuelo().getOrigen()
vueloDestino	String	reserva.getVuelo().getDestino()

■ Agrega un método estático desde(Reserva reserva) en ReservaResponseDTO que construya y retorne el DTO a partir de una entidad. Es el mismo patrón que OrdenResponseDTO que viste en clase.

04 ReservaService y ReservaController

El service de Reserva necesita tres repositorios: el propio, el de Pasajero y el de Vuelo — porque antes de guardar una reserva hay que buscar los objetos reales en la base de datos a partir de los IDs que llegaron en el DTO.

ReservaService — crear en el paquete service

Método	Recibe	Lógica	Retorna
findAll()	—	Trae todas las reservas, las convierte a DTO con stream + map	List
findById(Long id)	id	Busca la reserva. Si no existe retorna null. Si existe retorna el DTO	ReservaResponseDT 0
save(ReservaRequestD TO dto)	DTO de entrada	Busca el Pasajero por dto.getPasajeroId(). Busca el Vuelo por dto.getVueloId(). Construye la entidad Reserva y la guarda. Retorna el DTO de salida.	ReservaResponseDT 0
update(Long id, ReservaRequestDT O dto)	id + DTO	Busca la reserva existente. Si no existe retorna null. Actualiza fechaReserva, clase, pasajero y vuelo. Guarda y retorna DTO.	ReservaResponseDT 0
delete(Long id)	id	Elimina por ID	void

ReservaController — crear en el paquete controller

Anotado con @RestController y @RequestMapping("/reservas"). Inyecta ReservaService por constructor con @Autowired.

Anotación	Método del service que llama	Tipo de retorno
@GetMapping	findAll()	List
@GetMapping("/{id}")	findById(id)	ReservaResponseDTO
@PostMapping	save(dto)	ReservaResponseDTO
@PutMapping("/{id}")	update(id, dto)	ReservaResponseDTO

<code>@DeleteMapping("/{id}")</code>	<code>delete(id)</code>	<code>void</code>
--------------------------------------	-------------------------	-------------------

Probar Reserva en Postman

Primero asegurarse de tener al menos un pasajero y un vuelo creados.

```
POST http://localhost:8080/reservas
{
  "fechaReserva": "2026-06-01T09:00:00",
  "clase": "ECONOMICA",
  "pasajeroId": 1,
  "vueloId": 1
}
```

Respuesta esperada:

```
{
  "id": 1,
  "fechaReserva": "2026-06-01T09:00:00",
  "clase": "ECONOMICA",
  "pasajeroId": 1,
  "pasajeroNombre": "Laura Martínez",
  "vueloId": 1,
  "vueloOrigen": "BOG",
  "vueloDestino": "MDE"
}
```

✓ GET /reservas responde sin StackOverflowError — los DTOs están funcionando

05 ResponseEntity — códigos HTTP correctos

Hacer **GET /vuelos/999** en Postman. La respuesta es **200 OK** con cuerpo vacío. Eso está mal — el recurso no existe y la API responde como si todo estuviera bien. **ResponseEntity** es la clase que da control sobre el código HTTP que sale.

Código	Significado	Cuándo usarlo
200 OK	<code>ResponseEntity.ok(objeto)</code>	findAll, findById y update exitosos
201 Created	<code>ResponseEntity.status(HttpStatus.CREATED).body(obj)</code>	save exitoso
204 No Content	<code>ResponseEntity.noContent().build()</code>	delete exitoso
404 Not Found	<code>ResponseEntity.notFound().build()</code>	findById, update o delete cuando no existe

Actualizar los tres controllers

El patrón es el mismo en los cuatro controllers. Aquí está la estructura para VueloController — replicar en PasajeroController y ReservaController.

Método	Lógica con ResponseEntity
<code>obtenerTodos()</code>	Retornar <code>ResponseEntity.ok(service.findAll())</code>
<code>obtenerPorId(id)</code>	Si <code>service.findById(id)</code> es null → <code>notFound().build()</code> . Si no → <code>ok(resultado)</code>
<code>crear(body)</code>	Retornar <code>ResponseEntity.status(CREATED).body(service.save(body))</code>
<code>actualizar(id, body)</code>	Si <code>service.update()</code> retorna null → <code>notFound().build()</code> . Si no → <code>ok(resultado)</code>
<code>eliminar(id)</code>	Llamar <code>service.delete(id)</code> y retornar <code>noContent().build()</code>

- El tipo de retorno de cada método del controller cambia a `ResponseEntity`. Por ejemplo: `ResponseEntity` en `obtenerTodos()`, `ResponseEntity` en `obtenerPorId()`. Vuelo y Pasajero no tienen DTO — usan la entidad directamente como T.

✓ GET /vuelos/999 responde 404 · POST /vuelos responde 201 · DELETE responde 204

06 Bean Validation

Intentar **POST /vuelos** con origen vacío y fechaHora nula. La app crea el vuelo sin resistencia. Bean Validation permite declarar las reglas directamente en los campos — Spring las ejecuta automáticamente cuando recibe el request.

Dependencia — agregar al pom.xml

Bean Validation **no viene con spring-boot-starter-web**. Hay que agregar esta dependencia dentro del bloque `<dependencies>`:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Guardar y recargar Maven (ícono del elefante en IntelliJ).

Anotaciones que usas hoy

Anotación	Qué valida	Tipo de campo
<code>@NotBlank</code>	String no nulo, no vacío, no solo espacios	<code>String</code>
<code>@NotNull</code>	Campo no nulo	Cualquier tipo
<code>@Min(n)</code>	Número mayor o igual a n	<code>int</code> , <code>double</code> , <code>Integer</code> , <code>Double</code>

<code>@Email</code>	Formato de email válido	<code>String</code>
<code>message = "..."</code>	Mensaje personalizado en el error	Dentro de cualquier anotación

Vuelo.java — validaciones a agregar

Campo	Anotaciones
<code>origen</code>	<code>@NotBlank(message = "El origen no puede estar vacío")</code>
<code>destino</code>	<code>@NotBlank(message = "El destino no puede estar vacío")</code>
<code>fechaHora</code>	<code>@NotNull(message = "La fecha y hora del vuelo son obligatorias")</code>
<code>estado</code>	<code>@NotNull(message = "El estado del vuelo es obligatorio")</code>

Pasajero.java — validaciones a agregar

Campo	Anotaciones
<code>nombre</code>	<code>@NotBlank(message = "El nombre no puede estar vacío")</code>
<code>apellido</code>	<code>@NotBlank(message = "El apellido no puede estar vacío")</code>
<code>documento</code>	<code>@NotBlank(message = "El documento es obligatorio")</code>
<code>email</code>	<code>@NotBlank(message = "El email es obligatorio") · @Email(message = "El email no tiene un formato válido")</code>

ReservaRequestDTO — validaciones a agregar

Campo	Anotaciones
<code>fechaReserva</code>	<code>@NotNull(message = "La fecha de reserva es obligatoria")</code>
<code>clase</code>	<code>@NotNull(message = "La clase del asiento es obligatoria")</code>
<code>pasajeroId</code>	<code>@NotNull(message = "El id del pasajero es obligatorio")</code>
<code>vueloId</code>	<code>@NotNull(message = "El id del vuelo es obligatorio")</code>

Activar las validaciones en los controllers

Las anotaciones declaran las reglas. `@Valid` le dice a Spring que las ejecute. Sin `@Valid` las reglas existen pero nunca se evalúan. Agregar `@Valid` solo en los métodos que reciben body: `@PostMapping` y `@PutMapping`.


```
// Ejemplo en VueloController
@PostMapping
public ResponseEntity<Vuelo> crear(@Valid @RequestBody Vuelo vuelo) { ... }
@PutMapping("/{id}")
public ResponseEntity<Vuelo> actualizar(@PathVariable Long id,
                                       @Valid @RequestBody Vuelo datos) { ... }
```

■ @Valid va antes de @RequestBody, no después. El import es jakarta.validation.Valid.

Probar las validaciones

```
POST http://localhost:8080/vuelos
{ "origen": "", "destino": "MDE", "fechaHora": "2026-06-01T08:00:00", "estado": "PROG
RAMADO" }
→ Esperado: 400 Bad Request
POST http://localhost:8080/pasajeros
{ "nombre": "Laura", "apellido": "Martínez", "documento": "10203040", "email": "esto-
no-es-email" }
→ Esperado: 400 Bad Request
POST http://localhost:8080/reservas
{ "clase": "ECONOMICA", "pasajeroId": 1, "vueloId": 1 }
→ Esperado: 400 Bad Request (falta fechaReserva)
```

✓ Datos inválidos rechazados con 400 antes de llegar a la base de datos

07 Swagger / OpenAPI

Swagger genera automáticamente una interfaz visual con todos los endpoints de la API. Otro desarrollador puede leer la documentación, ver los modelos esperados y probar los endpoints directamente desde el navegador — sin necesidad de Postman.

Dependencia — agregar al pom.xml

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.5.0</version>
</dependency>
```

■ springdoc-openapi versión 2.x es compatible con Spring Boot 3.x. Si la app usa Spring Boot 2.x, la versión correcta es 1.7.0 y el artifactId cambia.

Sin escribir una línea más

Guardar, recargar Maven y reiniciar la app. Abrir en el navegador:

```
http://localhost:8080/swagger-ui.html
```

Spring detecta la dependencia y genera la documentación automáticamente. Todos los controllers, endpoints y modelos aparecen sin ninguna configuración adicional.

Opcional — mejorar la documentación con anotaciones

@Tag agrupa todos los endpoints de un controller bajo un nombre legible. **@Operation** describe qué hace un endpoint específico. Agregar al menos en VueloController como práctica:

```
@Tag(name = "Vuelos", description = "Gestión de vuelos de la aerolínea")
@RestController
@RequestMapping("/vuelos")
public class VueloController {
    @Operation(summary = "Listar todos los vuelos")
    @GetMapping
    public ResponseEntity<List<Vuelo>> obtenerTodos() { ... }
    @Operation(summary = "Buscar vuelo por ID")
    @GetMapping("/{id}")
    public ResponseEntity<Vuelo> obtenerPorId(@PathVariable Long id) { ... }
}
```

✓ Swagger UI accesible en /swagger-ui.html con los tres controllers documentados

Criterio de cierre

Al terminar debes poder demostrar todo esto en Postman y en el navegador:

- ✓ Tabla vuelo, tabla pasajero y tabla reserva creadas en pgAdmin con las columnas correctas
- ✓ CRUD completo de Vuelo: POST responde 201, GET responde 200, PUT actualiza, DELETE responde 204
- ✓ CRUD completo de Pasajero: mismo comportamiento
- ✓ POST /reservas con pasajeroId y vueloId válidos → respuesta limpia con nombre del pasajero y datos del vuelo
- ✓ GET /vuelos/999 → 404 Not Found (no 200 con cuerpo vacío)
- ✓ POST /vuelos con origen vacío → 400 Bad Request
- ✓ POST /pasajeros con email inválido → 400 Bad Request
- ✓ http://localhost:8080/swagger-ui.html muestra los tres controllers con sus endpoints

Cheat sheet

Estado final del proyecto

```

model/
  EstadoVuelo.java
  Vuelo.java           ← @NotBlank, @NotNull en los campos
  Pasajero.java        ← @NotBlank, @Email en los campos
  ClaseAsiento.java    ← enum nuevo
  Reserva.java         ← @ManyToOne a Pasajero y Vuelo
repository/
  VueloRepository.java
  PasajeroRepository.java
  ReservaRepository.java ← nuevo
service/
  VueloService.java    ← CRUD completo
  PasajeroService.java ← CRUD completo
  ReservaService.java  ← nuevo · usa ReservaRequestDTO / ReservaResponseDTO
controller/
  VueloController.java ← ResponseEntity + @Valid + @Tag
  PasajeroController.java ← ResponseEntity + @Valid
  ReservaController.java ← nuevo · ResponseEntity + @Valid
dto/
  ReservaRequestDTO.java ← @NotNull en todos los campos
  ReservaResponseDTO.java ← método estático desde(Reserva)

```

ResponseEntity — constructores de uso frecuente

Código	Cómo se escribe
200 OK con body	<code>ResponseEntity.ok(objeto)</code>
201 Created con body	<code>ResponseEntity.status(HttpStatus.CREATED).body(objeto)</code>
204 No Content	<code>ResponseEntity.noContent().build()</code>
404 Not Found	<code>ResponseEntity.notFound().build()</code>

Errores frecuentes

Error	Causa probable	Solución
Las validaciones no se disparan — entra cualquier dato	Falta <code>@Valid</code> en el parámetro del controller	Agregar <code>@Valid</code> antes de <code>@RequestBody</code> en POST y PUT
<code>StackOverflowError</code> al consultar reservas	Entidad expuesta directamente con relaciones circulares	Usar <code>ReservaResponseDTO</code> en lugar de la entidad
500 al crear reserva	<code>pasajeroId</code> o <code>vueloId</code> no existe en la BD	Crear primero el pasajero y el vuelo
Swagger UI no carga	Spring Boot 3.x con versión 1.x de <code>springdoc</code>	Verificar que la versión sea 2.5.0

NoSuchBeanDefinitionException al arrancar	Falta @Service o @Repository en alguna clase	Verificar anotaciones en ReservaService y ReservaRepository
Columna clase guarda 0, 1, 2 en la BD	Falta @Enumerated(EnumType.STRING)	Agregar la anotación en el campo clase de Reserva